

Revisão do Dia 2

Booleanos

>>> # Operadores de Comparação

>>> 4 <= 10

True

>>> "A" > "a"

False

>>> 'Olá!' == 'Olá!'

True

>>> 10 != 10

False

>>> # Operadores Lógicos

>>> (4 <= 10) and ("A" > "a")

False

>>> ('Olá!' == 'Olá!') or (10 != 10)

True

>>> not 6 > 7 and 4 == 2 and 3 > -2

False

>>> not (6 > 7 and 4 == 2) and 3 > -2

True

revisao_dia_2.py x

Estruturas de Seleção

>>> # if e else

>>> x: int = int(input("-> "))

-> -3

>>> if x > 0:

... print("Positivo")

... elif x == 0:

... print("Zero")

... else:

... print("Negativo")

...

Negativo

>>> # match case

>>> x: int = int(input("-> "))

-> 2

>>> match x:

... case 1:

... print("Caso um")

... case 2:

... print("Caso dois")

... case _:

... print("Caso padrão")

...

Caso dois

dia_3.py x

Dia 3 -- Listas e Estruturas de Repetição

'''

Nós já vimos como armazenar um dado de um determinado tipo em variáveis, porém como faríamos para armazenar vários dados de forma eficiente?

Vamos dizer que temos um aluno, e na matéria de física ele realizou 3 provas, e guardaremos a nota dessas provas da seguinte forma:

```
>>> joseNota1: float = 6.7
```

```
>>> joseNota2: float = 3.4
```

```
>>> joseNota3: float = 8.3
```

Isso parece meio contraintuitivo, especialmente quando podemos acabar errando o nome de uma das variáveis ou tiver outra com o nome parecido

'''



listas.py x

Listas

'''

Uma forma de resolvermos esse problema é com listas!

O conceito de arranjos ou "arrays" são estruturas sequenciais de dados de um mesmo tipo onde cada elemento são representados por índices

Normalmente arrays possuem um tamanho fixo, mas no Python, temos a estrutura "list", um arranjo dinâmico que, conforme elementos são adicionados ou removidos, a lista muda de tamanho.

'''

listas.py x

```
1
2 '''
3 Listas no Python são representadas com elementos entre colchetes, com cada
4 elemento separado por uma vírgula, e pode ser declarada da seguinte forma:
5 '''
6 >>> nome: list[dado]
7 >>> nome: list[dado] = [dado0, dado1, dado2]
8
9 '''
10 Uma lista declarada sem a inicialização, cria uma lista vazia ([]) onde
11 podemos adicionar elementos nela
12 Nosso exemplo anterior, com a estrutura de listas ficaria da seguinte forma:
13 '''
14 >>> notasJose: list[float] = [6.7, 3.4, 8.3]
15
```


listas.py x

```
1
2 # Indexação
3 '''
4 Como dito anteriormente, as listas são indexadas, e cada um desses índices
5 pode ser acessado, assim como nas strings!
6 '''
7 >>> nomes: list[str] = ["Jamily", "Anna Lisia", "Ana Julia"]
8 >>> nomes[0]
9 "Jamily"
10 >>> # Assim como nas strings, podemos pegar uma sublista
11 >>> nomes[1:]
12 ["Anna Lisia", "Ana Julia"]
13 >>> nomes[-1]
14 "Ana Julia"
15
```

listas.py x

1

2

3 # Operações e métodos

4 >>> # Assim como as strings, temos as operações de concatenação e repetição

5 >>> # Concatenação

6 >>> [2, 3, 5] + [1, 4]

7 [2, 3, 5, 1, 4]

8

9 >>> # Repetição

10 >>> [5, 6, 7] * 3

11 [5, 6, 7, 5, 6, 7, 5, 6, 7]

12

13

14

15



listas.py x

1 # Métodos

2 >>> # .append(item)

3 >>> # Adiciona *item* ao final da lista

4 >>> a: list[int] = [3, 6, 8]

5 >>> a.append(9)

6 >>> a

7 [3, 6, 8, 9]

8
9 >>> # .pop()

10 >>> # Remove o último elemento da lista e retorna o seu valor

11 >>> a.pop()

12 9

13 >>> a

14 [3, 6, 8]

15

listas.py x

```
1 >>> # .insert(pos, item)
2 >>> # Insere *item* na posição *pos*. Empurra todos os elementos seguintes
3 >>> # para frente
4 >>> a.insert(1, 2)
5 [3, 2, 6, 8]
6
7 >>> # .remove(item)
8 >>> # Remove a primeira ocorrência de *item*
9 >>> a.remove(3)
10 [2, 6, 8]
11
12 >>> # len(lista)
13 >>> # Retorna o numero de elementos da lista
14 >>> len(a)
15 3
```



listas.py x

```
1
2
3
4 >>> # .reverse()
5 >>> # Inverte a ordem dos elementos da lista
6 >>> a.reverse()
7 [8, 6, 2]
8
9 >>> # .sort()
10 >>> # Deixa a lista em ordem crescente
11 >>> a.sort()
12 [2, 6, 8]
```

listas.py x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

'''

Agora que vimos como funcionam as listas, vamos voltar ao exemplo das notas!

Digamos que eu queira fazer a média aritmética da nota de José:

```
>>> (notasJose[0] + notasJose[1] + notasJose[2]) / 3
```

```
6.1333333
```

Para esse caso é fácil visto que são apenas 3 valores, mas digamos que queremos a média da turma inteira com base nas 3 provas, em uma turma com apenas 20 alunos, isso já seriam mais de 60 elementos!

Definitivamente não é uma boa alternativa somar tudo isso manualmente...

'''



repeticao.py x

1

2 # Estruturas de Repetição

3

4 '''

5 A forma de solucionarmos esse problema é simples, vamos utilizar os famosos

6 laços de repetição!

7

8 Amplamente utilizados para realizar ações repetidas de forma eficiente, seja

9 com um número determinado de ciclos ou com uma condição específica de parada.

10

11 Serão abordados 2 tipos importantes de laço, o **for** e o **while**.

12 '''

13

14

15

repeticao.py x

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

Laço for

'''

No geral é usado quando queremos realizar um número determinado de repetições de uma mesma ação, ou percorrer todos os elementos de uma lista

>>> for i in lista:

... * bloco de código *

Traduzido vagamente pra “para **i** em **lista**”, quer dizer que para cada elemento da lista escolhida, armazena esse elemento em uma variável e executa o bloco de código

'''

repeticao.py x

1

2

3 # Exemplo

4 >>> cesta_frutas: list[str] = ["Uva", "Maçã", "Melancia", "Banana"]

5 >>> for fruta in cesta_frutas:

6 ... print(fruta)

7 ...

8 Uva

9 Maçã

10 Melancia

11 Banana

12

13

14

15

repeticao.py x

'''

>>> for i in range(x, y):

... * bloco de código *

Traduzido vagamente pra “para *i* no intervalo [x,y)”. Isso quer dizer que *i* receberá como valor todos os números de *x* até *y* (excluindo o *y*), e para cada um, ele executará o bloco de código

'''

Exemplo

>>> soma: int = 0

>>> for i in range (0, 6):

... soma += i

...

>>> soma

15

repeticao.py x

'''

>>> for i in range(x, y):

... * bloco de código *

Traduzido vagamente pra “para *i* no intervalo [*x*,*y*)”. Isso quer dizer que *i* receberá como valor todos os números de *x* até *y* (excluindo o *y*), e para cada um, ele executará o bloco de código

'''

Exemplo

>>> soma: int = 0

>>> for i in range (0, 6):

... soma += i

...

>>> soma

15

repeticao.py x

```
1
2 '''
3 0 intervalo de range é composto por três elementos o inicio, o fim, e o passo
4 com que é atualizada a variável
5
6 Logo, um range pode ser descrito da seguinte forma range(inicio, fim, passo).
7 Apenas o fim é realmente necessário de declarar range(fim), nesse caso,
8 assume-se que o início é 0, e o passo é 1
9
10 Algumas possibilidades de range são:
11 >>> range(10) # Vai de 0 a 10 com passo 1
12 >>> range(0, 10, 2) # Vai de 0 a 10, de 2 em 2 números (0,2,4,8)
13 '''
```



repeticao.py x

1

2

Laço while

3

4

'''

5

Usado geralmente para repetir blocos sobre uma condição

6

7

>>> while condição:

8

... * bloco de código *

9

10

Pode ser traduzido para "enquanto condição", assim, o bloco de código é

11

executado indefinidamente enquanto a condição é verdadeira (true)

12

'''

13

14

15

repeticao.py x

```
1
2 # Exemplo
3 >>> num: int = int(input("Insira um número: "))
4 >>> mult: int = 2
5 >>> achou_divisor: bool = False
6 >>> while not achou_divisor:
7 ...     if num % mult == 0:
8 ...         achou_divisor = True
9 ...         if mult == num:
10 ...             print("Não é primo")
11 ...         else:
12 ...             print("É primo")
13 ...     mult += 1
14
15
```

Lista de Compras

'''

A mãe de João pediu para ele ir as compras, porém ele acha que escrever a lista no papel é algo muito ultrapassado. Pensando nisso, ele te contratou pra que desenvolva um programa onde ele possa criar uma lista de compras, adicionar um item para comprar, removê-lo caso tenha sido comprado, e procurar por itens, além de exibir todos os que estão atualmente na lista!

Faça o programa usando os conteúdos que vimos nas últimas aulas, de forma que o usuário possa escolher qual ação ele deseja realizar, e que continue executando enquanto o usuário quiser.

'''